Implementation of Physics-informed ML Methods

1 Introduction

- This document covers multiple physics-informed ML methods as applied to a common dataset.
- The focus here is on solving inverse problems.
- An effort has been made to create the simplest and most prototypical example for each method.
- The intended application of each method is different.
  - Indirect measurement is used to replace an unmeasured time-variant parameter.
  - PINNs are used to expand model results to out of the testing domain.
  - Delta learning is used to improve model performance and capture lost physics.
  - Informed structure is used to improve model performance.

2 Dataset and ML models

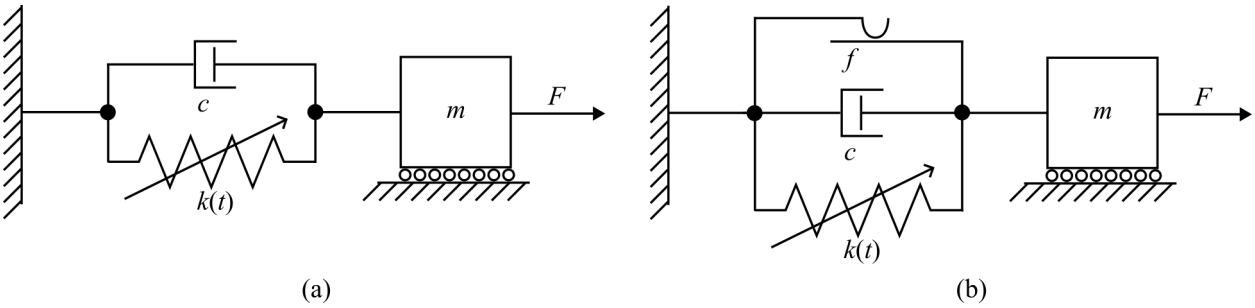

Figure 1: Mechanical system used throughout this work, showing the spring-mass-damper system: (a) without friction, and (b) with friction.

- The example continued through this paper is a one-dimensional inverse problem.
- Two mechanical systems are used, as shown in Fig. 2. They are spring-mass-damper systems with and without a friction damper added. One with the ideal governing system as shown in 2.1 and another representing the concept of lost physics with the introduction of a friction force, shown in 2.2

$$m\ddot{x} + c\dot{x} + k(t)x = F(t) \tag{2.1}$$

$$m\ddot{x} + c\dot{x} + k(t)x + f(\dot{x}) = F(t) \tag{2.2}$$

- $m = 1$ kg, $c = 0.2$ Ns/m, 5 Hz, ± 1 N sinusoidal forcing. Friction force $f(\dot{x})$ follows a Stribeck curve with a maximum static friction of 0.05 N.
- The systems are solved in Simulink/Simscape to generate the datasets.

- Over 100, 120 s tests, $k(t)$ ranges from 1500 N/m to 500 N/m following different paths as shown in Figure 2. Each path has two intermediate vertices between 1500 N/m and 500 N/m.

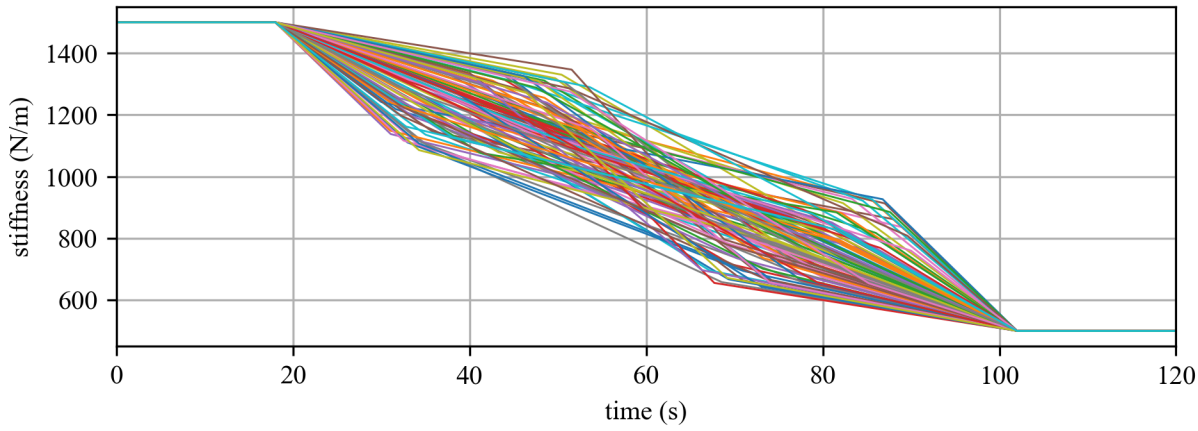

Figure 2: $k(t)$ for 100 tests.

- For this system, the damped harmonic frequency is 5 Hz when $k \approx 1000$ N/m. This produces resonance with a beat near the middle of the test such as seen in Fig. 2.

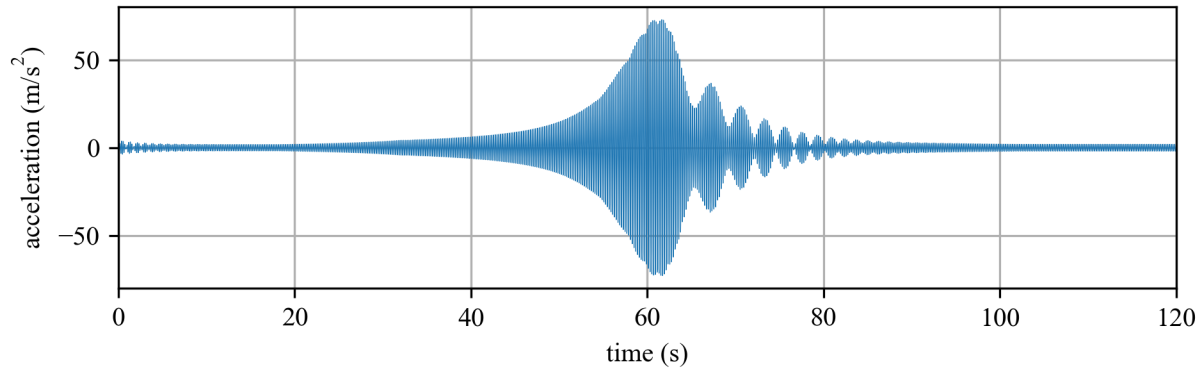

Figure 3: Measured acceleration for one test.

- From both datasets, 80 tests were taken for training and 20 for validation.
- To allow comparison between methodologies, a standard model is used. Unless stated otherwise, a dense neural network (DNN, NN or multilayer perceptron (MLP)), with four layers and hidden connections of shape [100, 100, 100, 1] was used. The first 3 layers use sigmoid activations while the last layer has an identity activation function (i.e. no activation function).
- The Adam optimizer with learning rate 0.001, $\beta_1 = 0.9$, $\beta_2 = 0.999$ (default tensorflow values recommended in the original paper) was used. Unless stated otherwise, the model was trained for 20 epochs.

3 Pure physics implementation

- Using windows of 1 second of data, a parameter fitting method (scipy.optimize.curve_fit) was used to estimate $k(t)$, assuming it is constant within the window.
- As a purely physics-informed method, no training data is needed.
- Downsides: This involves solving a nonlinear optimization problem at every step, which is expensive. A solution is only generated at the end of each window, introducing delay which may not be acceptable for real-time applications. It may be poor to assume that k is locally constant within each window may.

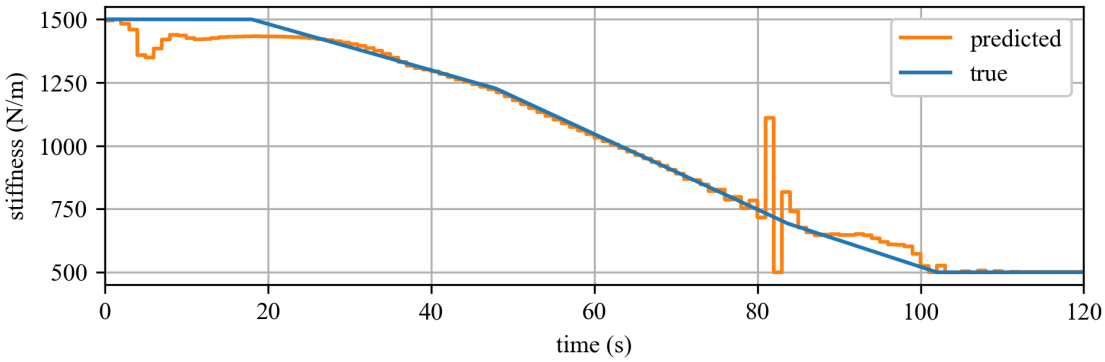

Figure 4: Physics prediction of one test.

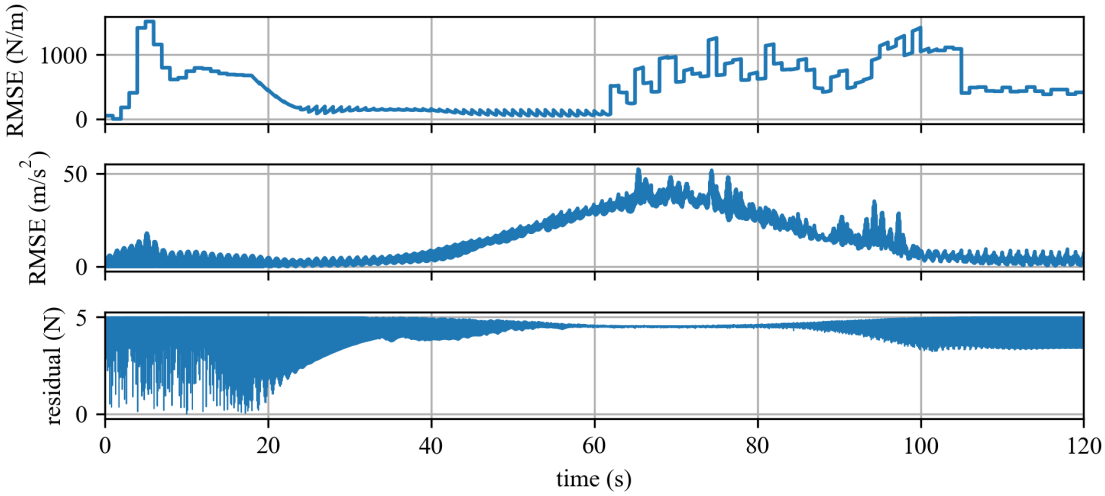

Figure 5: Data of pure physics prediction with respect to time, cumulative to all tests: showing; (a) RMSE error of reconstructed stiffness k; (b) RMSE error of reconstructed acceleration; and; (c) residual amount of lost physics.

4 Data-driven implementation

- This method uses a sliding window of 50 consecutive acceleration measurements as input to a neural network to predict the next k value. It is then trained end-to-end by comparing the predicted output

with the true k in a mean squared error objective.

- Mathematically, this is expressed as $\hat{k}(t+1) = F\big(a(t),\ a(t-1),\ \ldots,\ a(t-49)\big)$ where $(\hat{k}(t+1))$ is the *predicted* value of k at time $t+1$. $(a(\tau))$ is the acceleration at time τ. $F(\cdot)$ is the function learned by the neural network, mapping 50 consecutive acceleration measurements to the next k value.
- Data was downsampled to 20 S/s.
- 80 tests are taken for the training set and the remaining 20 are used for validation.
- A four-layer [100,100,100,1] model with sigmoid activations, optimized with Adam.

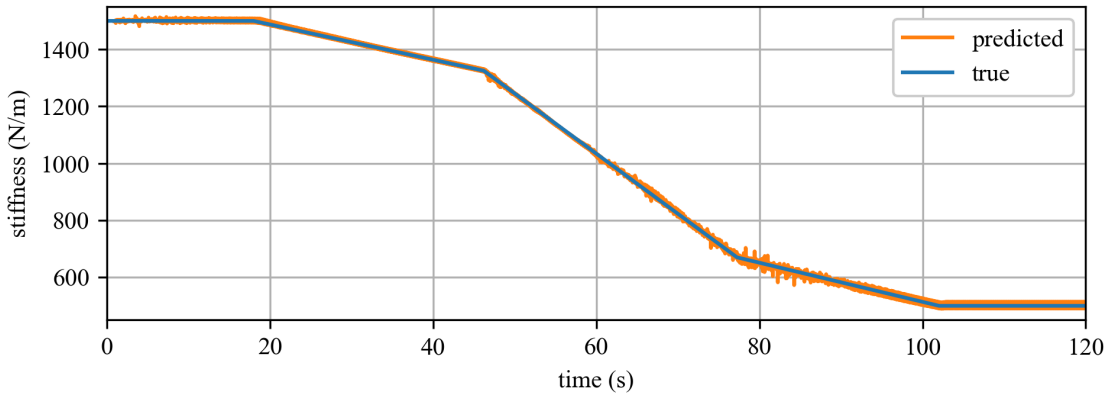

Figure 6: Neural network prediction of one test.

5 Methodology #1: indirect measurement/hard constraint

- An ODE solver is implemented within the automatic differentiation engine.
- An ML model produces an estimate stiffness, $\tilde{k}(t)$.
- This method is called a hard-constraint method since the output acceleration $\ddot{x}(t)$ is forced to obey the governing equation.

$$m\ddot{x} + c\dot{x} + \tilde{k}(t)x = F(t) \tag{5.1}$$

- Coble 2024 terms this indirect measurement since $k(t)$ is not required to generate $\tilde{k}(t)$ [1].

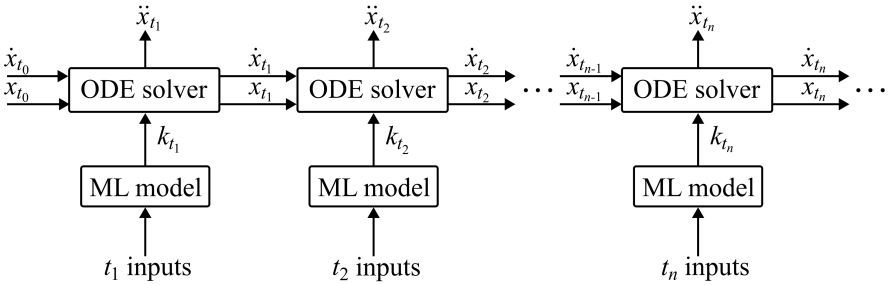

Figure 7: Hard constraint inference method. For this work, the chosen ODE solver is RK4 and t_i inputs are 50 points from the acceleration signal over the previous second.

- A mean-value filter over the last two cycles was used as a final layer of the machine learning model. This was found to be necessary after training without it resulted in a model which produced drastically different values within one cycle, as when x is near zero, the value of k has little impact on the output. The addition of the filter corresponds to the physical understanding that limits the rate of change of k.

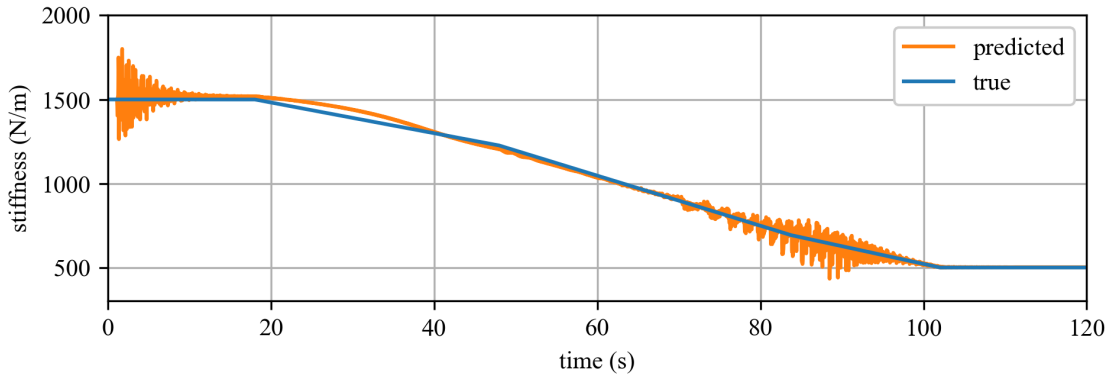

Figure 8: Indirect measurement prediction of $k(t)$ for one test. Even with the mean-value filter, ringing still occurs at the beginning and from 70-100 seconds in the test.

6 Methodology #2: PINNs/soft constraint

- PINNs are typically used for solving forward problems but can be used for inverse problems [2]. They leverage autodifferentiation to bring the governing equation directly into the loss function. The PINN was trained on a test without friction as it works best when the entire physics of the system is known.
- As PINNs are typically used for interpolation/forward problems, each trained PINN is specific to one test.
- Another essential element of PINN is the use of autodifferentiation to solve for terms out of the governing equation. As that is not needed for this problem (the governing equation does not contain derivatives of k), we have chosen to demonstrate the functionality through prediction of the variable x.
- The PINN estimates incrementing one timestep, as if replacing an ODE solver. If it is represented by the function ϕ, then it is

$$\phi : (t, x_{t-\Delta t}, v_{t-\Delta t}, F_t) \rightarrow (\tilde{x}_t, \tilde{k}_t) \tag{6.1}$$

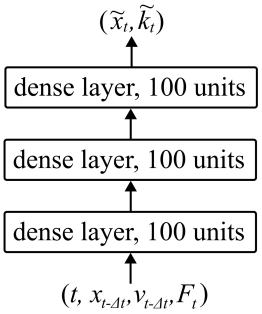

Figure 9: Model architecture for PINN.

- For notational simplicity, $\phi_1 = \tilde{x}_t$, $\phi_2 = \tilde{k}_t$, $\psi_1(t) = x_{t-\Delta t}$, $\psi_2(t) = v_{t-\Delta t}$, and $\psi_3(t) = F_t$. Then the first total derivative with respect to time is shown in equation 6.2 and the second total derivative is shown in equation 6.3. For the case here, only $i = 1$, the time derivatives of displacement, are needed.
- The PINN needs full access to the exact solution, including the highest derivatives. This may require numerically integrating or differentiating from experimental data.

$$\frac{d\phi_i}{dt} = \frac{\partial \phi_i}{\partial t} + \sum_{j=1}^{3} \frac{\partial \phi_i}{\partial \psi_j} \frac{d\psi_j}{dt} \tag{6.2}$$

$$\frac{d^2\phi_i}{dt^2} = \frac{\partial^2 \phi_i}{\partial t^2} + \sum_{j=1}^{3} \left(2 \frac{\partial^2 \phi_i}{\partial t \partial \psi_j} \frac{d\psi_j}{dt} + \frac{\partial \phi_i}{\partial \psi_j} \frac{d^2\psi_j}{dt^2} \right) + \sum_{j=1}^{3} \sum_{k=1}^{3} \frac{\partial^2 \phi_i}{\partial \psi_j \partial \psi_k} \frac{d\psi_j}{dt} \frac{d\psi_k}{dt} \tag{6.3}$$

- In equations 6.2 and 6.3, the partial derivatives on the right hand side are quantities found through autodifferentiation while the total derivatives are found experimentally or through numerical differentiation of experimental results.
- The PINN methodology uses two training sets: an experimental training set, notated $T_e = \{t_i\}_{i=1}^{N}$ and a physics training set $T_p = \{t_i\}_{i=1}^{M}$. The experimental set contains data measured from experiment, in this case stiffness values. Loss is represented as

$$L_k = \frac{1}{N} \sum_{t \in T_e} (k - \tilde{x}_t)^2 \tag{6.4}$$

$$L_x = \frac{1}{M} \sum_{t \in T_p} (x_t - \tilde{x}_t)^2 \tag{6.5}$$

$$L_{\dot{x}} = \frac{1}{M} \sum_{t \in T_p} \left(\dot{x}_t - \dot{\tilde{x}}_t \right)^2 \tag{6.6}$$

$$L_{\ddot{x}} = \frac{1}{M} \sum_{t \in T_p} \left(\ddot{x}_t - \ddot{\tilde{x}}_t \right)^2 \tag{6.7}$$

$$L_p = \frac{1}{M} \sum_{t \in T_p} \left(F_t - m\ddot{x}_t - c\dot{x}_t - \tilde{k}_t x_t \right)^2 \tag{6.8}$$

$$L = \rho_k L_k + \rho_x L_x + \rho_{\dot{x}} L_{\dot{x}} + \rho_{\ddot{x}} L_{\ddot{x}} + \rho_p L_p \tag{6.9}$$

- where ρ values are weighting constants chosen as hyperparameters.
- T_e was taken to be the first 60 s of the test, and T_p is the entire test.

Table 1: RMSE comparison between PINN and control model.

hyperparameter	value
ρ_k	0.01
ρ_x	100
$\rho_{\dot{x}}$	100
$\rho_{\ddot{x}}$	0.1
ρ_p	1.0

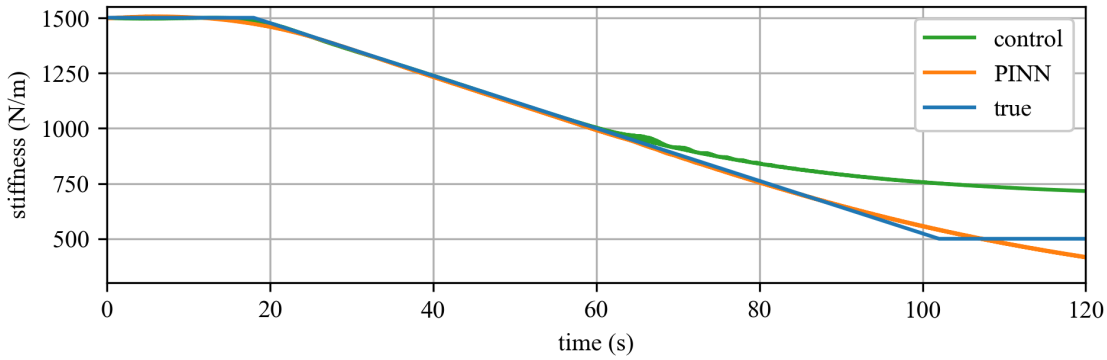

Figure 10: PINN prediction and control comparison.

Table 2: RMSE comparison between PINN and control model.

	PINN	control
0-60 s	6.51	2.06
60-120 s	119.25	81.71

- As seen in Fig. 10, the PINN ...

7 Methodology #3: Delta Learning

- With Delta learning, an assumption is made that the system can be simulated, though maybe not entirely accurately. Delta learning is done in a two step process.
- In this example, the frictionless dataset is used as the partially-inaccurate simulated data. To show improved domain performance, only the first half of each test in the friction tests was used, so that data with low stiffness is not included.
- Model 1 is the first step of delta learning, trained only on simulated data. The combined model is the second step of delta learning. Finally, the control model is a neural network trained only on the first half of the friction dataset.

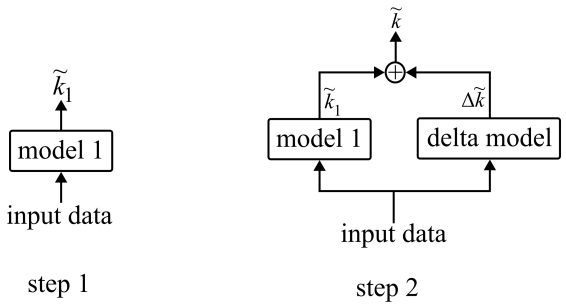

Figure 11: Model architecture for (a) model 1 and (b) delta learning model.

Table 3: RMSE comparison between initial and combined model.

	model 1	combined model	control
0-60 s	-	-	-
60-120 s	-	-	-

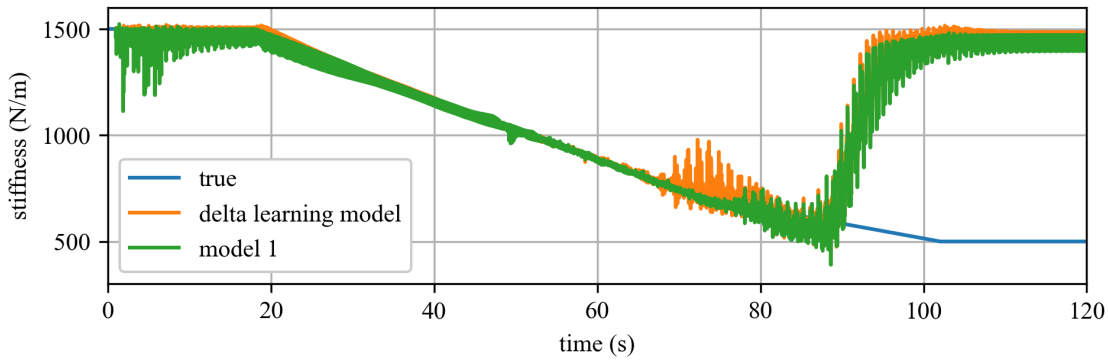

Figure 12: Prediction of delta learning model for one test.

- Looking at the jump in the prediction for Fig. 12 after 90 s. It seems like the model 1, trained on the frictionless dataset, associates the 100-120 s region of the with-friction dataset as being closer frictionless data which in near the beginning of the test. We do not know why this is or how to fix it.

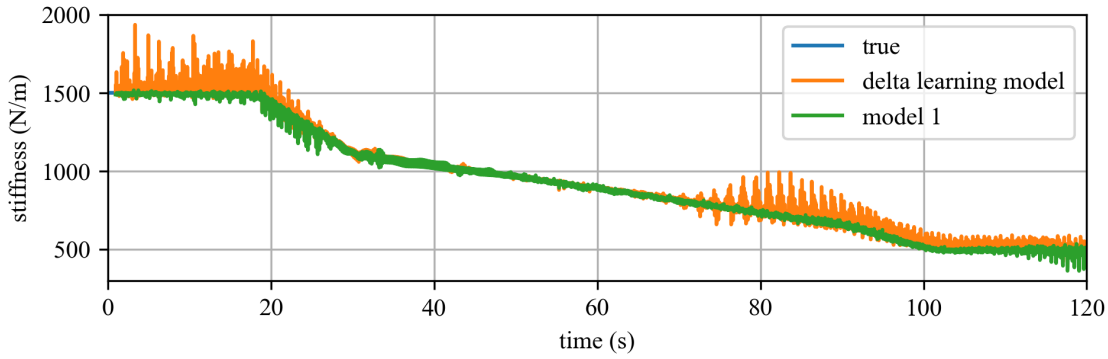

Figure 13: Prediction of delta learning model for one test of the no friction dataset.

- If you run this model on the on data set without friction, (Fig. 13 we do not get that jump at the end.

8 Methodology #4: Informed architecture

- An informed architecture uses a physical understanding in the broadest sense in informing the structure of a neural network architecture. That is, data exhibits symmetry and a model is implemented with this same symmetry. In so doing, the function space of the NN model is constrained to obey some aspect of physical understanding.
- The most common symmetry of data is time invariance. Two neural network architectures which handle time invariance are convolutional neural networks and recurrent neural networks.
- CNNs as weight-sharing of DNNs.
- This structure was chosen through hyperparameter tuning, but rather to make the size of the neural network, measured in number of weights, most closely match the pure data driven approach shown in 4 (25,420 vs. 25,401 respectively). This provides a fair comparison between the methodologies, although another method of comparison would look at computational efficiency.

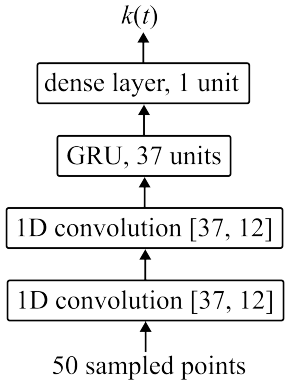

Figure 14: Model architecture of the informed structure model.

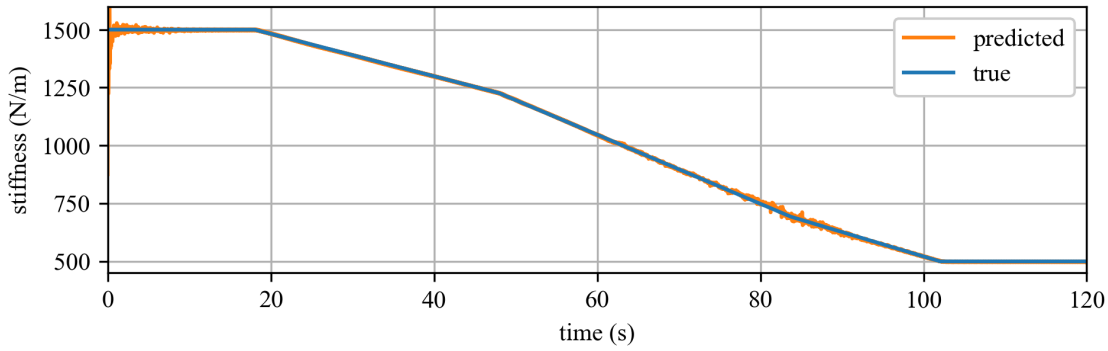

Figure 15: Informed structure prediction of one test.

9 Comparison

- The table below contains metrics for each model. Models are not all validated on the same datasets, so results cannot necessarily be compared directly against one another.

Table 4: Metric comparison between all models.

	SNR_{dB}	RMSE	MAE	TRAC
Pure physics	24.62	62.36	27.32	0.996
Pure NN	44.10	6.547	4.926	0.999
Indirect measurement	11.34	284.1	180.0	0.928
PINN	34.60	19.81	12.53	0.999
Delta learning	8.111	412.5	194.9	0.889
Informed architecture	39.17	11.58	3.410	0.999

- Pure NN outperforms informed architecture. Really, pure NN does very good.

References

[1] D. Coble, L. Cao, A. R. Downey, and J. M. Ricles, "Physics-informed machine learning for dry friction and backlash modeling in structural control systems," *Mechanical Systems and Signal Processing* **218**, p. 111522, Sept. 2024.

[2] M. Raissi, P. Perdikaris, and G. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics* **378**, pp. 686–707, Feb. 2019.